

1. Librerías externas y *frameworks*

1.1. Herramientas para trabajar con librerías

Las aplicaciones JavaScript complejas suelen utilizar numerosas **librerías**, por lo que se hace necesario utilizar **herramientas para su gestión**. En proyectos pequeños es posible utilizar simplemente etiquetas `<script>` y servidores CDN, pero proyectos más complejos exigen contar con herramientas que ofrezcan las siguientes **funcionalidades**:

- **Descarga** automática de librerías desde un repositorio centralizado o un CDN.
- Utilización de librerías creadas para **NodeJS** que no son directamente compatibles con el sistema de módulos ES6 del navegador.
- Gestión e instalación de **dependencias**.
- **Carga** de plantillas y componentes desde **diferentes archivos**.
- Utilización de tecnologías que no son directamente compatibles con los navegadores, como el lenguaje **TypeScript**, **CSS dinámico**, **sistemas de plantillas** o **componentes almacenados en archivos independientes**.
- **Empaquetado**, **generación** o **compilación** de una aplicación que no utiliza tecnologías compatibles con el navegador. Este proceso **genera archivos HTML, JavaScript y CSS** estándar listos para ser alojados en servidores web y utilizarse en producción.
- Utilización de **servidores de desarrollo** para poder realizar **pruebas** sobre la aplicación sin tener que realizar procesos de compilación.
- **Automatización** de procesos como minificación de archivos CSS, ejecución de test o compilación de la aplicación.

Existen numerosas herramientas creadas para realizar estas tareas. La mayoría de ellas están basadas en **paquetes NPM** ejecutables sobre el entorno **NodeJS**. Muchas de ellas se utilizan como **herramientas ejecutables** independientes de las dependencias del proyecto, por lo que se suelen instalar **a nivel de sistema**. Como alternativa, NodeJS incluye también la herramienta **npmx**, que permite **lanzar una aplicación NodeJS sin necesidad de instalarla** previamente en el sistema: **npmx** comprueba en primer lugar si el programa está instalado en la carpeta de proyecto o como paquete de sistema, y en caso contrario se conecta a Internet, busca el programa correspondiente y lanza su ejecución.

A continuación, se indican algunas de las **herramientas más utilizadas**, agrupadas por función:

A. GESTIÓN DE PAQUETES

- Se utilizan para instalar y cargar dependencias: librerías, ficheros externos, etcétera.
 - **NPM**. Es el **gestor de paquetes de NodeJS**. Aunque no es directamente compatible con el navegador, existen varios **empaquetadores** que permiten la utilización de paquetes NPM para proyectos de entorno cliente.
 - **Yarn**. Gestor de paquetes alternativo a NPM, creado por Facebook.
 - **Bower**. Gestor de paquetes **específico para el desarrollo *front-end***. Su tarea consiste en descargar la librería deseada y guardarla en una carpeta de dependencias del proyecto. En la actualidad está **abandonado** y sus creadores proponen el uso de Yarn junto con Webpack en su lugar.

B. EMPAQUETADO

- Generan archivos HTML, JavaScript y CSS estándar a partir de dependencias que utilizan distintos tipos de tecnologías, compatibles o no directamente con el navegador.
 - **Webpack**. Es la opción más utilizada hoy en día [2022]. Dada su versatilidad, se ha convertido en la herramienta ***de facto*** para ***frameworks*** como Angular, React o Vue. Proporciona también funcionalidades de **automatización**, por lo que su uso, junto con NPM, ha desplazado a otras herramientas, como Bower, Grunt o Gulp.

- **Browserify**. Similar a Webpack: permite la utilización de paquetes NPM, que utilizan el sistema de módulos de NodeJS y la generación de archivos empaquetados compatibles con el navegador.
- **RequireJS**. También similar a Webpack y Browserify, aunque más antigua.

C. AUTOMATIZACIÓN

- Se utilizan para tareas repetitivas que pueden realizarse de manera automática, como la ejecución de test, el empaquetado o la minificación de código CSS o JavaScript.
 - **Grunt**. Utiliza **ficheros de configuración**.
 - **Gulp**. Utiliza **código**, en lugar de ficheros de configuración.

1.2. Webpack

Webpack es la herramienta más utilizada en la actualidad para trabajar con proyectos de desarrollo en entorno cliente de cierta complejidad. Es utilizado por los principales *frameworks* JavaScript para desarrollar, testear y construir aplicaciones tipo SPA.

Su principal función es la de detectar todas las dependencias y **generar una aplicación compatible** con el navegador que utilice únicamente HTML, JavaScript y CSS. Así, Webpack permite trabajar con diferentes **sistemas de módulos** (como CommonJS, el más utilizado por NPM), **CSS dinámico** (SASS y LESS), **lenguajes** como TypeScript o CoffeeScript y **archivos de componentes** de React, Angular o Vue.

Una de las ventajas de utilizar Webpack es que **evita cargar ficheros de librerías a través de etiquetas <script>** en el objeto global. Este problema, ya solucionado con el sistema de módulos ES6, sigue estando presente en multitud de librerías antiguas. La carga de librerías a través de etiquetas **<script>** crea una cierta «**magia**» en el código de la aplicación, ya que los objetos proporcionados por la librería en cuestión pasan a estar **disponibles** a través del **objeto global** sin tener **ninguna referencia** de su carga en JavaScript (la única referencia está en el archivo HTML). Además, si existe alguna **dependencia** entre las librerías, se producirá un error si se incluyen los **<script>** en el **orden equivocado**.

Webpack es una herramienta **compleja**, por lo que no se pretende estudiarla en profundidad. Además, en la mayoría de los casos, su configuración será realizada de manera automática por las propias herramientas del *framework* específico que estemos utilizando (React, Angular o Vue).

Un proyecto de Webpack es un **proyecto de NodeJS**. Para crear un proyecto básico hay que ejecutar:

```
# Creación de la carpeta de proyecto
mkdir proyecto-webpack
cd proyecto-webpack
# Creación del proyecto de NodeJS con valores por defecto
npm init -y
# Instalación de Webpack como dependencia de desarrollo del proyecto
npm install webpack webpack-cli --save-dev
```

Dentro de la carpeta del proyecto se creará el fichero **package.json** con la configuración del proyecto. La configuración por defecto de Webpack exige la creación de **dos ficheros**: el archivo **index.html**, que es el archivo HTML de entrada a la aplicación, y el archivo **src/index.js**, que será el archivo JavaScript que almacenará el programa principal. La estructura de ficheros del proyecto quedará de la siguiente manera:

```
proyecto-webpack
|- package.json
|- package-lock.json
|- node-modules/
|- index.html
|- /src
   |- index.js
```

Para utilizar una librería, se deberá instalar como **dependencia** del proyecto mediante **npm**. Así, para instalar, por ejemplo, la dependencia de la librería **chart.js**, habrá que ejecutar:

```
npm install --save chart.js
```

npm instalará la librería correspondiente junto con sus dependencias dentro de la carpeta **node_modules** del proyecto. Si dentro del programa principal se desea hacer uso de la librería, se deberá **importar** mediante la palabra **import**. En el caso de la librería **chart.js**, se hará de la siguiente manera (según las instrucciones de dicha librería):

```
import Chart from 'chart.js/auto';
```

Como puede verse, la sentencia **import** no hace referencia a una ruta (es decir, no comienza por **./** o por **./node-modules/...**). Tal como estudiamos en la unidad correspondiente, **import** en el navegador necesita una referencia a una ruta; sin embargo, en este caso estamos tratando el proyecto como un proyecto de NodeJS, por lo que simplemente hay que indicar el nombre de la librería: Webpack se encargará de buscarla en la ruta adecuada, siguiendo las convenciones de los proyectos de NodeJS. Cuando Webpack realice el proceso de empaquetado para generar la aplicación, se encargará de resolver las rutas adecuadas.

En este punto, es necesario tener claro que la aplicación **no funcionará** si se carga el archivo **index.html**. El motivo, en este caso concreto, es sencillo: el proceso de carga de la librería **chart.js** no se ha realizado de manera compatible con el navegador, ya que **no se ha utilizado una ruta**. En aplicaciones más complejas, habrá seguramente más motivos: se habrá utilizado TypeScript, que deberá ser compilado previamente a JavaScript; se habrá utilizado SASS, que deberá ser compilado a CSS, o se habrán utilizado archivos **.vue**, que deberán ser traducidos a archivos HTML y CSS convencionales.

Para poder ejecutar la aplicación, será necesario realizar el proceso de **empaquetado** o **compilado**. En este proceso, se ejecutará el programa **webpack**, que se encargará de **generar la aplicación funcional** en la carpeta **/dist** dentro de la carpeta del proyecto (en el caso de utilizar la configuración por defecto). Para ello, se ejecutará el siguiente comando dentro de la carpeta del proyecto:

```
npx webpack
```

Este comando realiza las siguientes **tareas** por defecto:

1. **Busca las dependencias** utilizadas por el archivo **src/index.js** y el resto de archivos que haya importado.
2. **Genera** un archivo denominado por defecto **dist/main.js**, que es el archivo de salida. Este archivo utiliza exclusivamente **JavaScript compatible con el navegador**.
3. **Minimiza** el código generado para que ocupe menos espacio.

El último paso consistirá en **modificar** el archivo **index.html** para que utilice el archivo de salida **dist/main.js** mediante una etiqueta **<script>**. Esta será la única etiqueta **<script>** que se utilizará en el HTML. De esta manera, sí que podremos lanzar la aplicación a través del archivo **index.html**.

Para terminar, resulta de interés conocer los siguientes **detalles** sobre Webpack:

- Como hemos podido comprobar, es necesario realizar un proceso de **compilación** cada vez que se realice un **cambio en el código fuente**. Para evitar tener que ejecutar **npx webpack** cada vez que se realice un pequeño cambio, es posible utilizar un **servidor de desarrollo**. En la documentación de Webpack, puede consultarse más información al respecto.
- Webpack genera un **script convencional**, no de tipo módulo, para garantizar la **compatibilidad** con navegadores antiguos. Por ello, es recomendable **posicionar la etiqueta <script src=“./dist/main.js”>** al final de la etiqueta **<body>** del archivo **index.html**, para garantizar que el programa se ejecute cuando se haya **finalizado la carga** de todo el código HTML de la página.
- Es posible cambiar el nombre de los ficheros de entrada y salida a través del **fichero de configuración de Webpack**.
- En proyectos que utilicen tecnologías más complejas (SASS, TypeScript, etcétera), es posible que se genere **más de un archivo** en la carpeta **/dist**. En esos casos, habrá que consultar la documentación de Webpack para determinar qué archivos se deben incluir en **index.html** (por ejemplo, en el caso de utilizar algún sistema de generación de código CSS habrá que incluir como mínimo una etiqueta **<link rel=“stylesheet”>** para cargar el archivo de salida de CSS).

1.3. Frameworks JavaScript

El objetivo de los *frameworks* JavaScript es ofrecer una serie de funcionalidades para **agilizar y facilitar el desarrollo** de aplicaciones de entorno cliente. Los *frameworks* son **librerías software** que proporcionan **soluciones** a algunos de los **problemas y requisitos** que se presentan con **más frecuencia** en el desarrollo de estas aplicaciones. Algunas de las características que ofrecen son las siguientes:

- Un **sistema de plantillas** que permite utilizar una sintaxis **declarativa** similar a la utilizada en la programación en entorno servidor.
- Organización y división de la aplicación en **módulos o componentes**. Dichos componentes pueden estar creados mediante **lenguajes diferentes** a JavaScript, como **TypeScript** o **JSX**, o utilizar una **estructura específica**, como los archivos **.vue**.
- Herramientas específicas para la **creación y compilación** o **empaquetado** de la aplicación, con funciones de **generación de archivos y estructuras de carpetas**, **minificación** y **optimización de código**. Estas herramientas suelen utilizar a su vez herramientas genéricas como **Webpack** por debajo.
- **Servidor de desarrollo** integrado con soporte *hot reload* o **recarga automática**, que permite **visualizar los cambios** que se produzcan en el código fuente **sin necesidad** de realizar el proceso de compilación.
- *Data binding* o **enlace de datos**, que posibilita que la **interfaz se actualice** de manera **automática** cuando se produzca un **cambio en los datos** de los que depende.
- Sistema de **enrutamiento**, que permite **cargar una vista o un componente** en función de la **URL** a la que se esté accediendo.
- Herramientas para realizar **test**.

Muchas de las funciones que ofrecen los *frameworks* han sido incluidas en **versiones recientes** de JavaScript o en las API de navegador; es el caso de los **módulos, clases o componentes** (*web components* estándar). Sin embargo, muchas otras características, como el **enlace de datos**, siguen sin tener un equivalente estándar. Por ello, los *frameworks* JavaScript son **muy utilizados** en el desarrollo de aplicaciones de entorno cliente de **cierta complejidad** y sobre todo en el desarrollo de aplicaciones de tipo **SPA, single-page application**.

Los *frameworks* más utilizados en la actualidad son **Angular, React y Vue**. Cada uno de ellos ofrece su propio ecosistema de herramientas y tecnologías, pero las funciones que ofrecen son relativamente parecidas. Algunos de ellos incorporan más funcionalidad «de serie», como Angular, mientras que otros dependen más de librerías complementarias, como Vue. **No hay ninguno que sea claramente mejor**: la elección de uno u otro dependerá de diferentes factores no necesariamente relacionados con temas técnicos, como pueden ser el grado de **familiaridad** que tenga el **equipo de desarrollo** con un *framework* en concreto o la existencia de una **base de código** que utilice un *framework* determinado.

En los **siguientes apartados** de esta unidad estudiaremos las principales características de **Angular y React**, mientras que en la **siguiente unidad** estudiaremos el *framework* **Vue** en detalle.